

Day 85: SQL CRUD and Basic JOIN

1. Day Number

Day 85

2. Topic Name

INSERT, UPDATE, DELETE, and Basic JOIN

Today you will learn how to:

- add new rows with `INSERT`
 - change existing rows with `UPDATE`
 - remove rows with `DELETE`
 - combine related tables with a basic `INNER JOIN`
 - understand the basic idea of a **foreign key**
-

3. Connection

Yesterday you learned how to **read rows from a database** using:

```
SELECT
```

and filter those rows using:

```
WHERE
```

Today you will go one step further.

Instead of only reading data, you will learn how to **change data** and how to connect information stored in **two different tables**.

For example:

```
customers
```

may contain customer information, while:

```
orders
```

contains their purchases.

SQL can connect those tables using a `JOIN`.

4. Important Topics

```
INSERT
```

Adds a new row to a table.

```
INSERT INTO customers (customer_id, name, city)
VALUES (1, 'Asha', 'Bengaluru');
```

Think:

| "Create a new record."

UPDATE

Changes data that already exists.

```
UPDATE customers
SET city = 'Mysuru'
WHERE customer_id = 1;
```

Think:

| "Find this row and change something."

The **WHERE** condition is extremely important.

DELETE

Removes rows.

```
DELETE FROM customers
WHERE customer_id = 1;
```

Think:

| "Find this row and remove it."

Again, **WHERE** is important because without it, many or all rows could be deleted.

INNER JOIN

Combines matching rows from two tables.

For example:

```
customers
-----
customer_id | name
1           | Asha
2           | Ravi
```

and:

```
orders
-----
order_id | customer_id
101      | 1
102      | 2
```

The shared `customer_id` lets SQL connect each order to its customer.

Foreign Key Idea

A **foreign key** is a column that refers to an ID in another table.

For example:

```
customers.customer_id
```

might be the main ID of each customer.

Then:

```
orders.customer_id
```

can refer to that customer.

Conceptually:

```
customers
customer_id = 1
    ↑
    |
orders
customer_id = 1
```

This creates a relationship between the two tables.

5. Foundational Notes

A database often separates information into multiple tables instead of putting everything into one huge table.

For example, a `customers` table could contain:

```
customer_id | name | city
1           | Asha | Bengaluru
2           | Ravi | Chennai
```

An `orders` table could contain:

order_id	customer_id	product
101	1	Laptop
102	1	Mouse
103	2	Keyboard

Notice that the customer's name is not repeated inside every order.

Instead, `customer_id` connects the tables.

This reduces unnecessary repetition.

6. CRUD Explained Simply

CRUD describes four common things we do with stored data.

CRUD	Meaning	SQL command
Create	Add data	<code>INSERT</code>
Read	View data	<code>SELECT</code>
Update	Change data	<code>UPDATE</code>
Delete	Remove data	<code>DELETE</code>

You already learned the **Read** part yesterday.

So:

```
Create → INSERT
Read   → SELECT
Update → UPDATE
Delete → DELETE
```

A simple example could be:

```
INSERT INTO customers ...
SELECT * FROM customers;
UPDATE customers ...
DELETE FROM customers ...
```

These four operations appear constantly in database applications.

7. Why JOIN Is Useful

Imagine you want to answer:

| Which customer placed each order?

The `customers` table knows the customer's name.

customer_id	name
1	Asha
2	Ravi

The `orders` table knows the order.

order_id	customer_id	product
101	1	Laptop
102	2	Keyboard

Neither table alone contains everything you want.

A `JOIN` lets you combine them temporarily:

name	order_id	product
Asha	101	Laptop
Ravi	102	Keyboard

The original tables remain separate. SQL simply combines matching information when you query it.

8. Easy Example: Customers and Orders

Suppose we have these tables.

customers

customer_id	name	city
1	Asha	Bengaluru
2	Rahul	Mumbai
3	Meera	Delhi

orders

order_id	customer_id	product
101	1	Laptop
102	2	Phone
103	1	Mouse

We could add another customer:

```
INSERT INTO customers (customer_id, name, city)
VALUES (4, 'Neha', 'Pune');
```

We could update Rahul's city:

```
UPDATE customers
SET city = 'Hyderabad'
WHERE customer_id = 2;
```

We could remove a particular customer:

```
DELETE FROM customers
WHERE customer_id = 4;
```

And we could connect customers with orders:

```
SELECT customers.name, orders.product
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

The important part is:

```
ON customers.customer_id = orders.customer_id
```

It tells SQL:

| Match rows where the customer IDs are equal.

Possible result:

name		product
Asha		Laptop
Rahul		Phone
Asha		Mouse

Notice that Asha appears twice because she has two matching orders.

9. Problem Statement

You have two simple tables.

customers

customer_id		name		city
1		Anil		Delhi
2		Priya		Mumbai

orders

order_id	customer_id	product
201	1	Keyboard
202	2	Monitor
203	1	Mouse

Your task is to:

1. Add one new customer.
2. Update the city of one existing customer.
3. Think about how `customers` and `orders` can be connected.
4. Write a basic query that would show the customer's name together with the product they ordered.

Keep the exercise small.

10. Concepts Used

For this exercise, you will use:

- database tables
- rows and columns
- `INSERT INTO`
- `VALUES`
- `UPDATE`
- `SET`
- `WHERE`
- `SELECT`
- `INNER JOIN`
- `ON`
- customer IDs
- foreign-key relationships

11. Thought Process

Before writing SQL, think about what kind of operation you are performing.

If you want to **add** something:

```
INSERT
```

If you want to **find/read** something:

```
SELECT
```

If you want to **change** something:

```
UPDATE
```

If you want to **remove** something:

```
DELETE
```

If the information you need exists in **two related tables**:

```
JOIN
```

For the customer/order relationship, ask:

```
What column exists in both tables?
```

The answer is:

```
customer_id
```

Therefore that column can be used to connect the tables.

12. Beginner-Friendly SQL Steps

Step 1: Examine the customer table

Identify its columns:

```
customer_id  
name  
city
```

Step 2: Add a customer

Use the pattern:

```
INSERT INTO table_name (column1, column2, column3)  
VALUES (value1, value2, value3);
```

Replace the table, columns, and values with the appropriate customer information.

Step 3: Update one customer

Use:

```
UPDATE table_name  
SET column_name = new_value  
WHERE condition;
```

Make sure your `WHERE` identifies the correct customer.

Step 4: Identify the relationship

Look at both tables.

You should notice:

```
customers.customer_id
orders.customer_id
```

These values connect customers to their orders.

Step 5: Build a JOIN

The basic pattern is:

```
SELECT ...
FROM first_table
INNER JOIN second_table
ON first_table.common_column = second_table.common_column;
```

Choose the columns you want to display.

For this exercise, you probably want something similar to:

```
customer name
product
```

13. Easy Edge Cases

Missing Customer

Suppose an order contains:

```
customer_id = 50
```

but there is no customer `50` in the `customers` table.

An `INNER JOIN` will normally not return that order because there is no matching customer.

Customer Without an Order

Suppose you add:

```
3 | Rohan | Jaipur
```

to `customers`, but Rohan has never placed an order.

With an `INNER JOIN`, Rohan will not appear because there is no matching order.

Remember:

```
INNER JOIN → return rows that have matches in both tables
```

Later, you can learn other joins that can keep unmatched rows.

14. Expected Results

After your `INSERT`, the customer table should contain one additional row.

For example, conceptually:

```
Before:
2 customers

After INSERT:
3 customers
```

After your `UPDATE`, the selected customer's city should contain the new value.

```
Before:
Priya | Mumbai

After:
Priya | another city
```

Your JOIN result should look conceptually like:

```
name | product
Anil  | Keyboard
Priya | Monitor
Anil  | Mouse
```

The exact output depends on the data you use.

15. Hint Only

For adding the customer, start with:

```
INSERT INTO customers (...)
VALUES (...);
```

For updating the city, remember to include:

```
WHERE customer_id = ...;
```

For the JOIN, ask yourself:

| Which column connects an order to the customer who placed it?

Then use that relationship after:

```
ON ...
```

A useful skeleton is:

```
SELECT ...  
FROM customers  
INNER JOIN orders  
ON ...;
```

Try filling in the missing parts yourself.

Key lesson for Day 85: `SELECT` reads data, while `INSERT`, `UPDATE`, and `DELETE` change it. A `JOIN` lets you retrieve related information from multiple tables, usually by matching an ID such as `customer_id`.